GRADE 12 · TERM 1

SQL, Design & Data

A pencil-and-paper workbook: ask the table questions on paper first, learn the SQL, then design and defend your data.

Name: _____

Class: _____ Date: _____

How to use this workbook

1. Work with a **pencil**. Mistakes are how we learn: rub out and try again.
2. Start each unit with **Think First**. Try it on your own before you read the explanation.
3. Read **Learn It**, then do the activities. Trace tables and code are written **by hand**.
4. At the end of each unit, tick the "I can" boxes you are sure of. Write down anything you still want to ask.
5. The **checkpoint test** is done under test conditions, with no notes.

In this workbook

1. Asking a table questions
2. Smarter conditions
3. Calculating in SQL
4. Groups and links
5. Changing the data, and SQL in Delphi
6. Good database design on paper
7. Data theory in paragraphs
8. Term checkpoint test

> **Matches the Annual Teaching Plan:** Term 1 — SQL: SELECT, FROM, WHERE, ORDER BY, DISTINCT; calculated fields and aliases; LIKE, IN, BETWEEN, IS NULL and dates; SUM, MIN, MAX, AVG, COUNT, TOP; GROUP BY and HAVING; INSERT, UPDATE, DELETE; ROUND, INT, LEFT, RIGHT, MID, LEN, FORMAT; DAY, MONTH, YEAR, Date(); joining tables with WHERE; TADOQuery and parameterised queries. Theory: data collection, warehousing and mining; caring for data; characteristics of a good database; referential integrity; anomalies; normalisation (1NF to 3NF); mobile hardware and privacy; social implications of Big Data.

UNIT 1 Asking a table questions

Before you write any SQL you can already ask a table questions in plain English: show only these rows, put them in this order, count them. SQL is simply those questions written in a short, strict form that a database understands.

By the end of this unit I can: (tick each box when you are sure)

- ☐ name the parts of a query: SELECT, FROM, WHERE and ORDER BY
- ☐ turn a plain-English question about a table into SQL
- ☐ choose fields, filter rows and sort a result
- ☐ use DISTINCT to show each value once
- ☐ work out by hand which rows a query returns
- ☐ spot the usual mistakes in a query

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

1.1 Be the database

Here is the learner table of a small school. Use only a pencil and answer in plain English, as a database would.

tblLearner

LearnerID PK	Surname	Name	Grade	Sport	Cell	BirthDate	Mark
1	Dlamini	Thandi	12	Netball	0821234567	2007/03/14	78
2	Pillay	Ryan	11	Rugby	0839876543	2008/07/02	64
3	Mokoena	Naledi	12	Netball		2007/11/23	91
4	Botha	Anke	10	Hockey	0716543210	2009/01/30	55
5	Khumalo	Sipho	11	Soccer	0825551122	2008/05/19	82
6	Adams	Zaid	12	Soccer		2007/09/08	47
7	Naidoo	Priya	10	Netball	0794443321	2009/04/11	73
8	Jacobs	Liam	11	Rugby	0612228899	2008/12/05	69
9	Sithole	Lerato	12	Hockey	0845557788	2007/06/27	86
10	Petersen	Chante	10	Chess	0763330099	2009/08/16	60

(a) Write the surnames of the **Grade 12** learners.
(b) Write the surnames of the **netball** players in alphabetical order.
(c) How many learners play **hockey**?
(d) Write the sports that are played at the school, but write each sport **only once**.

LEARN IT

A query is a question

SQL (Structured Query Language) is the language we use to talk to a database, and a **query** is one question. Every part of the question has its own keyword, and the parts always come in the same order.

Keyword	Means	Compulsory?
SELECT	which fields (columns) to show	yes
FROM	which table to use	yes
WHERE	keep only the rows that pass a test	no
ORDER BY	sort the rows (ASC = small to big, the default; DESC = big to small)	no

A query ends with a semicolon. SQL does not care about capital letters, but we write the keywords in capitals so they stand out. Field names and table names must be spelled exactly as in the database.

SQL

```
SELECT Surname, Mark
FROM tblLearner
WHERE Grade = 12
ORDER BY Mark DESC;
```

LEARN IT

How the database reads your query

You **write** SELECT first, but the database **works** in a different order: it takes the table (**FROM**), throws away the rows that fail the test (**WHERE**), keeps only the columns you named (**SELECT**) and finally sorts what is left (**ORDER BY**). That is the order you would follow with a pencil and the paper table. Conditions in WHERE use **=**, **<>** (not equal), **<**, **>**, **<=** and **>=**. **Text goes in double quotes** and **numbers do not**. Use **SELECT *** to show every field. To sort by two fields, separate them with a comma: *ORDER BY Grade, Surname* sorts by grade and, inside each grade, by surname.

SQL

```
SELECT *
FROM tblLearner
WHERE Sport = "Hockey";

SELECT Surname, Grade
FROM tblLearner
ORDER BY Grade DESC, Surname;
```

LEARN IT

DISTINCT: each value once

The Sport column holds Netball three times. **SELECT Sport FROM tblLearner** would show it three times, because there is one row per learner. Put the word **DISTINCT** straight after SELECT to remove repeated rows from the result.

SQL

```
SELECT DISTINCT Sport
FROM tblLearner
ORDER BY Sport;
```

1.2 The words of a query [6]

A query that shows fields starts with _____. The table is named after _____. Only rows that pass a test are kept by _____, and the result is sorted by _____. In a condition, text is written inside _____ but numbers are not. Writing _____ after a field puts the biggest value first.

LEARN IT

Reference: tblLearner

`tblLearner`

LearnerID PK	Surname	Name	Grade	Sport	Cell	BirthDate	Mark
1	Dlamini	Thandi	12	Netball	0821234567	2007/03/14	78
2	Pillay	Ryan	11	Rugby	0839876543	2008/07/02	64
3	Mokoena	Naledi	12	Netball		2007/11/23	91
4	Botha	Anke	10	Hockey	0716543210	2009/01/30	55
5	Khumalo	Sipho	11	Soccer	0825551122	2008/05/19	82
6	Adams	Zaid	12	Soccer		2007/09/08	47
7	Naidoo	Priya	10	Netball	0794443321	2009/04/11	73
8	Jacobs	Liam	11	Rugby	0612228899	2008/12/05	69
9	Sithole	Lerato	12	Hockey	0845557788	2007/06/27	86
10	Petersen	Chante	10	Chess	0763330099	2009/08/16	60

1.3 Write it: Grade 11 [3]

Using the table in Think First 1.1, write a query that shows the **Surname** and **Sport** of every learner in **Grade 11**.

1.4 Write it: netball [4]

Show the **Surname** and **Mark** of the **netball** players, sorted by surname from A to Z. Write the result.

Surname	Mark

1.5 Write it: each sport once [3]

Write a query that lists the different sports, each **once only**, in alphabetical order.

1.6 Predict the rows [3]

Work through the query on the table above, then write the rows that appear.

SQL

```
SELECT Surname, Mark
FROM tblLearner
WHERE Sport = "Soccer"
ORDER BY Mark DESC;
```

Surname	Mark

1.7 Find the mistake [4]

Each query has **one** mistake. Say what it is.

SQL

```
1.  SELECT Surname, Mark tblLearner WHERE Grade = 12;
2.  SELECT Surname FROM tblLearner WHERE Sport = Netball;
3.  SELECT Surname, FROM tblLearner;
4.  SELECT Surname FROM tblLearner ORDER BY Mark DESC WHERE Grade = 12;
```

UNIT 2 Smarter conditions

One test is rarely enough. AND, OR and NOT combine tests; LIKE finds patterns; IN, BETWEEN and IS NULL are shortcuts for common tests; dates need their own symbol. You will practise on paper until you can predict exactly which rows come back.

By the end of this unit I can: (tick each box when you are sure)

- ☐ combine conditions with AND, OR, NOT and brackets
- ☐ explain why AND is worked out before OR
- ☐ use LIKE with the wildcards % and _
- ☐ use IN, BETWEEN and IS NULL correctly
- ☐ write a date in a query as #yyyy/mm/dd#
- ☐ predict the rows a query returns from a printed table

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

2.1 Sorting the learner cards

Use the learner table below. Write the **surnames** that match each description.

`tblLearner`

LearnerID PK	Surname	Name	Grade	Sport	Cell	BirthDate	Mark
1	Dlamini	Thandi	12	Netball	0821234567	2007/03/14	78
2	Pillay	Ryan	11	Rugby	0839876543	2008/07/02	64
3	Mokoena	Naledi	12	Netball		2007/11/23	91
4	Botha	Anke	10	Hockey	0716543210	2009/01/30	55
5	Khumalo	Sipho	11	Soccer	0825551122	2008/05/19	82
6	Adams	Zaid	12	Soccer		2007/09/08	47
7	Naidoo	Priya	10	Netball	0794443321	2009/04/11	73
8	Jacobs	Liam	11	Rugby	0612228899	2008/12/05	69
9	Sithole	Lerato	12	Hockey	0845557788	2007/06/27	86
10	Petersen	Chante	10	Chess	0763330099	2009/08/16	60

(a) Grade 12 **and** netball.
(b) Netball **or** hockey.
(c) A mark from 60 to 80, both numbers **included**.
(d) No cell number has been captured.
(e) Born in 2008.
(f) A surname that starts with S.

LEARN IT

AND, OR, NOT and brackets

AND: both tests must be true. **OR**: at least one test must be true. **NOT** reverses a test. SQL works out **AND before OR**, just as maths works out × before +. So a mixed condition that you read as "(A or B) and C" must be written with **brackets**, otherwise it means "A, or (B and C)". When in doubt, add brackets.

SQL

```
SELECT Surname
FROM tblLearner
WHERE (Sport = "Netball" OR Sport = "Hockey") AND Grade = 12;
```

LEARN IT

LIKE, IN, BETWEEN, IS NULL and dates

Test	Meaning	Example
LIKE	matches a pattern. **%** stands for any number of characters (even none); _ stands for exactly one character	`Surname LIKE "S%"` starts with S `Surname LIKE "%a"` ends in a

		`Surname LIKE "%ai%"` contains ai
IN	equal to any value in a list (a short way of writing several ORs)	`Grade IN (10, 12)`
BETWEEN	from a small value to a big one, both ends **included**	`Mark BETWEEN 60 AND 80`
IS NULL	the field is empty. Never write = NULL: it matches nothing, ever	`Cell IS NULL` `Cell IS NOT NULL`
#date#	a date is surrounded by # signs and written year first, so that it cannot be mixed up with day and month	`BirthDate >= #2008/01/01#`

Which wildcard? Use the wildcard character that the question's database uses (% or *). In this workbook, as in Delphi's ADO connection, we use **%** and _; Access's own query window uses ***** and **?** for the same jobs.

2.2 LIKE patterns [10]

Write the surnames in the table above that match each pattern. (The pattern is not case sensitive.)

Condition	Surnames that match
`Surname LIKE "S%"`	
`Surname LIKE "%a"`	
`Surname LIKE "_a%"`	
`Surname LIKE "%ai%"`	
`Surname LIKE "%o%o%"`	

2.3 Predict: without brackets [3]

Write the rows that appear.

SQL

```
SELECT Surname, Grade, Sport
FROM tblLearner
WHERE Sport = "Netball" OR Sport = "Hockey" AND
Grade = 12;
```

Surname	Grade	Sport

2.4 Predict: with brackets [4]

Only the brackets have changed. Write the rows that appear. Why is there one row fewer?

SQL

```
SELECT Surname, Grade, Sport
FROM tblLearner
WHERE (Sport = "Netball" OR Sport = "Hockey")
AND Grade = 12;
```

Surname	Grade	Sport

2.5 Predict: NULL and a date [3]

Write the rows that appear. A row appears if it passes **either** test.

```SQL
SELECT Surname, BirthDate
FROM tblLearner
WHERE Cell IS NULL OR BirthDate < #2007/07/01#;
```

Surname	BirthDate

2.6 Write it: two sports, two grades [4]

Show the **Surname**, **Grade** and **Sport** of learners in Grade 11 or 12 who play rugby or soccer. Use IN for the grades and brackets round the sports.

2.7 Write it: a date range [3]

Show the **Surname** and **BirthDate** of learners born from 1 January 2007 to 30 June 2007.

2.8 Why does it return nothing? [4]

Each of these queries runs without an error, yet returns **no rows** from tblLearner. Say why, and write the corrected condition.

```SQL
1.  WHERE Cell = NULL
2.  WHERE Mark BETWEEN 80 AND 60
3.  WHERE Surname LIKE "S"
4.  WHERE Grade = 10 AND Grade = 12
```

UNIT 3 Calculating in SQL

SQL can calculate as well as filter. It can work out a new column for every row, boil a whole column down to one number, and cut text and dates into pieces. In this unit you will find those answers by hand first, so you know what the database is doing.

By the end of this unit I can: (tick each box when you are sure)

- ☐ create a calculated field and name it with AS
- ☐ use SUM, MIN, MAX, AVG and COUNT to get one value from many rows
- ☐ use TOP n together with ORDER BY
- ☐ use ROUND and INT
- ☐ use LEN, LEFT, RIGHT, MID and FORMAT on text
- ☐ use YEAR, MONTH, DAY and Date() with dates
- ☐ work out a single-value result by hand

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

3.1 The shopkeeper's pencil

Mama Dube runs a spaza shop. Here is her stock list:

`tblProduct`

ProductID PK	ProdName	Category	Price	Stock
1	Brown bread	Bakery	16.50	30
2	Full-cream milk 1L	Dairy	19.99	18
3	Maize meal 5kg	Staples	54.90	12
4	Candles (6)	Load shedding	24.00	40
5	Eggs (dozen)	Dairy	49.50	9
6	Rechargeable lamp	Load shedding	129.00	0
7	Sugar 2kg	Staples	38.00	25
8	Matches	Load shedding	6.50	60

Answer with a pencil only.

(a) Which product has the **highest** price and which has the **lowest**?

(b) How many products cost **more than R20**?

(c) How many items are in stock **altogether**?

(d) What is the **average price** of a product, to the nearest cent? (Add the eight prices, then divide by 8.)

LEARN IT

Calculated fields and aliases

In the SELECT list you can write a calculation (with +, -, *, /) instead of a field name. The database works it out **for every row**. Give the new column a name with **AS**: that name is called an **alias**. Without an alias the column gets a meaningless name such as Expr1001. You can also join pieces of text with **&**. An alias exists only in the result, so you may not use it inside WHERE: repeat the calculation instead.

SQL

```sql
SELECT ProdName, Price * Stock AS StockValue
FROM tblProduct
ORDER BY StockValue DESC;

SELECT Surname & ", " & Name AS FullName
FROM tblLearner;
```

LEARN IT

Aggregate functions and TOP

An **aggregate function** boils many rows down to **one** value.

Function	Gives	Example
SUM(field)	the total	`SUM(Stock)`
MIN(field)	the smallest value	`MIN(Price)`
MAX(field)	the biggest value	`MAX(Price)`
AVG(field)	the average	`AVG(Mark)`
COUNT(*)	the number of rows	`COUNT(*)`

WHERE is worked out **first**, so the aggregate only looks at the rows that passed. A query with an aggregate may not also list an ordinary field (that needs GROUP BY, in the next unit). **TOP n** keeps only the first n rows. It is only meaningful with **ORDER BY**, because "first" depends on the sort order.

SQL

```sql
SELECT COUNT(*) AS NumDear
FROM tblProduct
WHERE Price > 20;

SELECT MIN(Price) AS Cheapest, MAX(Price) AS Dearest
FROM tblProduct;

SELECT TOP 3 ProdName, Price
FROM tblProduct
ORDER BY Price DESC;
```

LEARN IT

Functions for numbers, text and dates

A function takes a value in brackets and gives one back. You can use them in SELECT, in WHERE and inside other functions.

Function	What it does	Example	Gives
ROUND(x, n)	rounds to n decimal places	`ROUND(17.991, 1)`	18
INT(x)	keeps the whole part (rounds down)	`INT(17.991)`	17
LEN(text)	number of characters	`LEN("Naledi")`	6
LEFT(text, n)	the first n characters	`LEFT("Mokoena", 3)`	Mok
RIGHT(text, n)	the last n characters	`RIGHT("0821234567", 4)`	4567
MID(text, start, n)	n characters from position start (the first character is position 1)	`MID("0821234567", 4, 3)`	123
FORMAT(x, "R0.00")	shows a number in a set layout	`FORMAT(16.5, "R0.00")`	R16.50
YEAR / MONTH / DAY(date)	one part of a date as a number	`YEAR(#2007/03/14#)` `MONTH(#2007/03/14#)`	2007 3

		`DAY(#2007/03/14#)`	14
Date()	today's date	`YEAR(Date()) - YEAR(BirthDate)`	the age this year

3.2 The value of one answer [6]

Work out the single value that each query gives. Use the stock list in Think First.

Query	Result
`SELECT COUNT(*) FROM tblProduct WHERE Category = "Load shedding"`	
`SELECT SUM(Stock) FROM tblProduct`	
`SELECT MAX(Price) FROM tblProduct WHERE Category = "Dairy"`	
`SELECT MIN(Stock) FROM tblProduct WHERE Stock > 0`	
`SELECT ROUND(AVG(Price), 2) FROM tblProduct`	
`SELECT SUM(Price * Stock) FROM tblProduct WHERE Category = "Staples"`	

3.3 Write it: sale prices [4]

Write a query that shows the **ProdName** of every *Load shedding* product and a column called **SalePrice**: the price with **10% off**, rounded to 2 decimals.

3.4 Write it: name and age [5]

Today is 28 October 2026. For the netball players, show one column called **ShortName** (the first letter of the name, a full stop and space, then the surname, as in *T. Dlamini*) and a column called **Age** (this year minus the birth year).

3.5 Predict the function [7]

Work out what each expression gives for the stated learner or product (see the tables in Units 2 and 3).

Expression	For	Result
`LEN(Surname)`	the learner with LearnerID 3	
`LEFT(Sport, 3)`	LearnerID 7	
`RIGHT(Cell, 4)`	LearnerID 5	
`MID(Cell, 4, 3)`	LearnerID 5	
`INT(Mark / 10)`	LearnerID 3	
`MONTH(BirthDate)`	LearnerID 8	
`FORMAT(Price, "R0.00")`	ProductID 2	

3.6 Write it: the best three [3]

Show the **Surname** and **Mark** of the three learners with the highest marks.

UNIT 4 Groups and links

Sometimes you want an answer for every group: the number of learners in each sport, the sales of every product. GROUP BY does that. Then you will join two tables, so that a sale shows the name of the product instead of a number, and meet the most common mistake in a join.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain what GROUP BY does with a picture of piles of rows
- ☐ use GROUP BY with COUNT, SUM, AVG, MIN and MAX
- ☐ choose between WHERE and HAVING
- ☐ work out the result of a grouped query by hand
- ☐ join two tables with a WHERE condition on PK = FK
- ☐ explain the Cartesian-product trap

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

4.1 Piles of cards

Imagine each learner is a card. Put the cards for the same sport in one **pile**, then summarise every pile.

Surname	Sport	Mark
Dlamini	Netball	78
Pillay	Rugby	64
Mokoena	Netball	91
Botha	Hockey	55
Khumalo	Soccer	82
Adams	Soccer	47
Naidoo	Netball	73
Jacobs	Rugby	69
Sithole	Hockey	86
Petersen	Chess	60

Complete the summary of the piles. Write each sport once.

Sport	Cards in the pile	Total of the marks	Average mark (1 decimal)

Now cross out every pile with **fewer than 2** cards. Which sports are left?

LEARN IT

GROUP BY and HAVING

GROUP BY field puts the rows that have the same value in that field into one group, and the query gives **one result row per group**. Every field in the SELECT list must either appear in GROUP BY or be inside an aggregate function (COUNT, SUM, AVG, MIN, MAX). **HAVING** filters the **groups** after they have been made, so it is the place to test an aggregate. **WHERE** filters single **rows** before the groups are made, and can never contain an aggregate. Inside HAVING repeat the aggregate; do not use an alias. Access lists the groups in order of the GROUP BY field.

Order of work	Clause	What it does
1	FROM	take the table
2	WHERE	throw away single rows
3	GROUP BY	make the groups
4	HAVING	throw away whole groups
5	SELECT	work out the columns and aggregates
6	ORDER BY	sort the result

SQL

```
SELECT Sport, COUNT(*) AS Num, ROUND(AVG(Mark), 1) AS AvgMark
FROM tblLearner
WHERE Grade >= 11
GROUP BY Sport
HAVING COUNT(*) >= 2
ORDER BY Sport;
```

4.2 Predict: WHERE, then groups, then HAVING [5]

Follow the order of work: first throw away the rows that fail WHERE, then make the groups, then throw away the groups that fail HAVING. Write the result.

SQL

```
SELECT Sport, ROUND(AVG(Mark), 1) AS AvgMark
FROM tblLearner
WHERE Grade >= 11
GROUP BY Sport
HAVING COUNT(*) >= 2;
```

Sport	AvgMark

THINK FIRST · no computer needed

4.3 Matching the slips

The shop keeps its sales in a separate table. A sale only stores the **ProductID**, not the name.

tblProduct

ProductID PK	ProdName	Category	Price	Stock
1	Brown bread	Bakery	16.50	30
2	Full-cream milk 1L	Dairy	19.99	18
3	Maize meal 5kg	Staples	54.90	12
4	Candles (6)	Load shedding	24.00	40
5	Eggs (dozen)	Dairy	49.50	9
6	Rechargeable lamp	Load shedding	129.00	0
7	Sugar 2kg	Staples	38.00	25
8	Matches	Load shedding	6.50	60

tblSale

SaleID PK	ProductID	Qty	SaleDate
101	1	3	2026/10/01
102	4	2	2026/10/01
103	2	1	2026/10/02
104	4	5	2026/10/02
105	8	10	2026/10/03
106	1	2	2026/10/03
107	3	1	2026/10/04
108	6	1	2026/10/05
109	2	4	2026/10/05
110	4	1	2026/10/06

Use ProductID to find the product of each sale. For sales 101 to 105 write the **SaleID, product name, quantity and amount** (**quantity × price**). Draw a pencil line from each sale to its product as you go.

LEARN IT

Joining two tables with WHERE

The tables are linked by a **primary key and a foreign key**: ProductID is the primary key of tblProduct and a foreign key in tblSale. To see fields of both tables in one result, list both tables after **FROM**, separated by a comma, and write the link as a condition in **WHERE**: *foreign key = primary key*. Further conditions follow with AND. If a field name exists in both tables, put the table name in front (**tblSale.ProductID**), otherwise the query is ambiguous and fails. A short **table alias** written after the table name saves typing: *FROM tblSale S, tblProduct P* lets you write S.Qty and P.Price.

SQL

```
SELECT tblSale.SaleID, tblProduct.ProdName, tblSale.Qty
FROM tblSale, tblProduct
WHERE tblSale.ProductID = tblProduct.ProductID
AND tblProduct.Category = "Load shedding";

SELECT S.SaleID, P.ProdName, S.Qty * P.Price AS Amount
FROM tblSale S, tblProduct P
WHERE S.ProductID = P.ProductID;
```

LEARN IT

The Cartesian-product trap

If you list two tables and **forget the link**, the database pairs **every row of one table with every row of the other**. Here are a coach table and a team table:

tblCoach

CoachID PK	CoachName
1	Mr Adams
2	Mrs Botha

tblTeam

TeamID PK	TeamName	CoachID
1	U14 Netball	2
2	U16 Soccer	1
3	U15 Hockey	2

Without the link the query returns 2 × 3 = 6 rows, and most of them are false (Mr Adams does not coach the netball team):

CoachName	TeamName
Mr Adams	U14 Netball
Mr Adams	U16 Soccer
Mr Adams	U15 Hockey
Mrs Botha	U14 Netball
Mrs Botha	U16 Soccer
Mrs Botha	U15 Hockey

With *WHERE tblCoach.CoachID = tblTeam.CoachID* only the 3 true pairs remain:

CoachName	TeamName
Mrs Botha	U14 Netball
Mr Adams	U16 Soccer
Mrs Botha	U15 Hockey

Rule: the number of tables minus one is the number of link conditions you need.

4.4 Predict: a join with a condition [4]

Work out the rows (the tables are on this page). Write the result.

SQL

```
SELECT tblSale.SaleID, tblProduct.ProdName,
tblSale.Qty
FROM tblSale, tblProduct
WHERE tblSale.ProductID = tblProduct.ProductID
AND tblProduct.Category = "Load shedding"
ORDER BY tblSale.SaleID;
```

SaleID	ProdName	Qty

4.5 Predict: a join with GROUP BY [4]

First match each sale to its product, then group by category. Write the result.

SQL

```
SELECT P.Category, SUM(S.Qty) AS TotalSold
FROM tblSale S, tblProduct P
WHERE S.ProductID = P.ProductID
GROUP BY P.Category;
```

Category	TotalSold

4.6 Write it: money per product [6]

Write a query that shows each product name and the money it brought in (**Qty × Price** added up, called **Income**), but only for products that brought in **more than R100**, the biggest income first. Join the tables with WHERE and use the aliases S and P.

4.7 Find the mistake [3]

Each query has one fault. Say what is wrong.

SQL

```
1.  SELECT ProdName, Qty
    FROM tblSale, tblProduct
    WHERE tblProduct.Category = "Dairy";

2.  SELECT ProductID, ProdName
    FROM tblSale S, tblProduct P
    WHERE S.ProductID = P.ProductID;

3.  SELECT P.Category, SUM(S.Qty) AS TotalSold
    FROM tblSale S, tblProduct P
    WHERE S.ProductID = P.ProductID
    GROUP BY P.Category
    HAVING TotalSold > 4;
```

UNIT 5 Changing the data, and SQL in Delphi

SELECT only looks at data. INSERT, UPDATE and DELETE change it, and a missing WHERE can ruin a whole table. Then you will learn how Delphi sends SQL to the database through a TADOQuery, including a value typed by the user.

By the end of this unit I can: (tick each box when you are sure)

- ☐ predict the effect of INSERT, UPDATE and DELETE on a table
- ☐ explain why UPDATE and DELETE need WHERE
- ☐ write INSERT, UPDATE and DELETE statements
- ☐ choose Open or ExecSQL correctly
- ☐ build an SQL string that contains a typed value
- ☐ write a query with one parameter and run it in Delphi

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

5.1 Edit the paper table

Mama Dube changes her stock list with a pencil. Start with this table:

`tblProduct`

ProductID PK	ProdName	Category	Price	Stock
1	Brown bread	Bakery	16.50	30
2	Full-cream milk 1L	Dairy	19.99	18
3	Maize meal 5kg	Staples	54.90	12
4	Candles (6)	Load shedding	24.00	40
5	Eggs (dozen)	Dairy	49.50	9
6	Rechargeable lamp	Load shedding	129.00	0
7	Sugar 2kg	Staples	38.00	25
8	Matches	Load shedding	6.50	60

Make **three** changes, in this order. (1) **Add** a new product: Paraffin 1L, Load shedding, R21.50, 15 in stock (it gets the next ProductID). (2) **Add R2** to the price of every Staples product. (3) **Remove** every product that has no stock. Write the table as it looks afterwards.

ProductID	ProdName	Category	Price	Stock

LEARN IT

INSERT, UPDATE and DELETE

These three statements **change** the table and give **no rows back**. To see the effect you must run a SELECT afterwards. **INSERT INTO** names the table, lists the fields in brackets and then gives the **VALUES**

in the **same order**. An AutoNumber key is left out because the database fills it in. **UPDATE** changes the fields named after **SET**, on the rows chosen by WHERE. **DELETE FROM** removes the rows chosen by WHERE. **Always write WHERE** on UPDATE and DELETE: without it **every** row is changed or removed.

SQL

```
INSERT INTO tblProduct (ProdName, Category, Price, Stock)
VALUES ("Paraffin 1L", "Load shedding", 21.50, 15);

UPDATE tblProduct
SET Price = Price + 2
WHERE Category = "Staples";

UPDATE tblProduct
SET Price = 20, Stock = 5
WHERE ProductID = 2;

DELETE FROM tblProduct
WHERE Stock = 0;
```

5.2 Complete the sentences [6]

_____ adds a new row. _____ changes values that already exist, and the new values are given after the word _____. _____ removes rows. If the _____ part is left out, every row in the table is affected. These statements do not return rows, so to see the result you must run a _____ afterwards.

5.3 Predict: the missing WHERE [3]

The table has 8 rows. How many rows does each statement change or remove, and what does the table look like afterwards?

SQL

```
UPDATE tblProduct
SET Price = 10;

DELETE FROM tblProduct;
```

5.4 Write it: three statements [10]

Write the SQL for each change to **tblLearner** (LearnerID is an AutoNumber).
(a) **Add** a learner: Surname Mahlangu, Name Bongani, Grade 10, Sport Soccer, Cell 0791230000, born 12 February 2009, Mark 66.
(b) Petersen's mark should be 62, not 60: **correct** it.
(c) **Remove** every Grade 12 learner whose mark is below 50. How many learners does it remove from the table in Unit 2?

LEARN IT

TADOQuery: Close, SQL, Open

In Delphi, SQL is sent to the database by a **TADOQuery** component (we call it **qryLearner**). The steps are always the same: **Close** the query, put the SQL text in its **SQL** property, then run it. A long query is easier to read if you **Clear** the SQL and **Add** it line by line. Each Add is a separate line, so you do not need spaces between them (when you join pieces with + you do).

Delphi

```
qryLearner.Close;
qryLearner.SQL.Text := 'SELECT Surname, Mark FROM tblLearner WHERE Grade = 12 ORDER BY Mark
DESC';
qryLearner.Open;

qryLearner.Close;
qryLearner.SQL.Clear;
qryLearner.SQL.Add('SELECT Surname, Mark');
qryLearner.SQL.Add('FROM tblLearner');
qryLearner.SQL.Add('WHERE Grade = 12');
qryLearner.SQL.Add('ORDER BY Mark DESC');
qryLearner.Open;
```

LEARN IT

Open or ExecSQL?

A SELECT gives records back, so you run it with **Open** and then read the records. INSERT, UPDATE and DELETE give nothing back, so you run them with **ExecSQL**. Using Open on an UPDATE causes an error. To read the records of a query, use *qryLearner['Surname']* inside a loop. A query that returns one row, such as an aggregate, needs no loop.

Statement	Gives records back?	Run it with
SELECT	yes	**Open**
INSERT, UPDATE, DELETE	no	**ExecSQL**

Delphi

```
qryLearner.Close;
qryLearner.SQL.Text := 'UPDATE tblLearner SET Mark = 62 WHERE Surname = "Petersen"';
qryLearner.ExecSQL;

memOut.Clear;
qryLearner.First;
while not qryLearner.Eof do
begin
  memOut.Lines.Add(qryLearner['Surname'] + ': ' + IntToStr(qryLearner['Mark']));
  qryLearner.Next;
end;
```

LEARN IT

A value typed by the user

Way 1: build the string. Join the typed value into the SQL text with +. A number needs no quotes; text needs double quotes put round it. Watch the spaces where the pieces meet. **Way 2: a parameter** (preferred). Write a **:name** in the SQL and give it a value before you run the query. The parameter takes care of quotes by itself, and it also stops a user from typing SQL of their own into the box (SQL injection). You may use **one parameter** in this grade. For a LIKE query put the wildcard on the value: *.Value := edtSearch.Text + '%'*.

Delphi

```
// a number: no quotes round it
sSQL := 'SELECT Surname, Mark FROM tblLearner WHERE Grade = ' + edtGrade.Text;
// text: double quotes round it
sSQL := 'SELECT Surname, Mark FROM tblLearner WHERE Sport = "' + edtSport.Text + '"';
qryLearner.Close;
qryLearner.SQL.Text := sSQL;
qryLearner.Open;
// typed Rugby, the second string becomes: ... WHERE Sport = "Rugby"

qryLearner.Close;
qryLearner.SQL.Clear;
qryLearner.SQL.Add('SELECT Surname, Mark');
qryLearner.SQL.Add('FROM tblLearner');
qryLearner.SQL.Add('WHERE Grade = :pGrade');            // :pGrade is the parameter
qryLearner.Parameters.ParamByName('pGrade').Value := StrToInt(edtGrade.Text);
qryLearner.Open;
```

5.5 Open or ExecSQL? [6]

Circle the method you use to run each statement.

1	SELECT Surname FROM tblLearner	**Open** **ExecSQL**
2	INSERT INTO tblLearner (Surname) VALUES ("Zulu")	**Open** **ExecSQL**
3	UPDATE tblProduct SET Stock = 0	**Open** **ExecSQL**
4	SELECT COUNT(*) AS Num FROM tblLearner	**Open** **ExecSQL**
5	DELETE FROM tblSale WHERE SaleID = 110	**Open** **ExecSQL**
6	SELECT DISTINCT Sport FROM tblLearner	**Open** **ExecSQL**

5.6 Predict the SQL text [4]

The user types **Rugby** in edtSport and **65** in edtMin. (a) Write the exact text that ends up in sSQL. (b) Write the rows the query returns (use the table in Unit 2).

Delphi

```
sSport := edtSport.Text;
iMin := StrToInt(edtMin.Text);
sSQL := 'SELECT Surname, Mark ' +
        'FROM tblLearner ' +
        'WHERE Sport = "' + sSport + '" ' +
        'AND Mark >= ' + IntToStr(iMin);
```

5.7 Debug: raise the marks [3]

This button should add 5 to the mark of every learner in the grade typed in edtGrade. Find **three** errors.

Delphi

```
procedure TfrmMain.btnRaiseClick(Sender: TObject);
begin
  qryLearner.SQL.Text := 'UPDATE tblLearner SET Mark = Mark + 5' +
                         'WHERE Grade = ' + edtGrade.Text;
  qryLearner.Open;
end;
```

5.8 Write it: show a grade **[12]**

edtGrade holds a grade. When **btnShow** is clicked, use a parameterised query on **qryLearner** (parameter **pGrade**) to list the learners of that grade, **highest mark first**, in **memOut**, each as *Surname: Mark*. Show the number of learners in **lblCount** (for example *4 learners*).

UNIT 6 Good database design on paper

A database that stores the same fact in many places breaks in three predictable ways. You will see the breaks happen on paper, learn the rule that stops them (referential integrity), and then repair a badly designed table by normalising it step by step. A normalisation question can be set in every exam.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain the three anomalies with an example
- ☐ explain referential integrity and spot an orphan record
- ☐ name the characteristics of a good database
- ☐ decide whether a table is in 1NF, 2NF and 3NF
- ☐ normalise a table to 3NF, writing every table with its keys
- ☐ draw the ERD of a normalised design

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

6.1 When one big table goes wrong

Mama Dube keeps every order in one table:

`tblOrderInfo`

OrderNo	OrderDate	CustCell	CustName	ProductCode	ProductName	UnitPrice	Qty
1001	2026/10/01	0821112233	Mrs Dube	B01	Brown bread	16.50	3
1001	2026/10/01	0821112233	Mrs Dube	C04	Candles (6)	24.00	2
1002	2026/10/01	0734445566	Mr Fortuin	B01	Brown bread	16.50	1
1003	2026/10/02	0821112233	Mrs Dube	M07	Maize meal 5kg	54.90	1
1003	2026/10/02	0821112233	Mrs Dube	C04	Candles (6)	24.00	4
1004	2026/10/03	0617778899	Ms Peters	E02	Eggs (dozen)	49.50	2
1004	2026/10/03	0617778899	Ms Peters	M07	Maize meal 5kg	54.90	2

(a) She buys a new product, Sugar 2kg, that nobody has ordered yet. Where in the table can she record it, and what must she make up?
(b) Order 1004 is cancelled and its rows are deleted. Which customer and which product disappear from the table completely?
(c) Mrs Dube gets a new cell number. How many rows must change?
(d) The price of candles goes up. How many rows must change? What happens if one is missed?

LEARN IT

The three anomalies

When the same fact is stored in many rows, three errors appear:

Anomaly	What goes wrong	In Think First
Insertion	you cannot record one fact without recording another unrelated fact	a new product has no order yet
Deletion	deleting a row also destroys a fact that you wanted to keep	cancelling order 1004 erases Ms Peters and the Eggs
Modification (update)	one fact is changed in some rows but not in others, so the data contradicts itself	a new cell number is typed in 3 of Mrs Dube's 4 rows

The cure is to store every fact **once**, in its own table, and to link the tables with keys.

LEARN IT

Referential integrity and a good database

A **foreign key** holds the primary key of a row in another table. **Referential integrity** is the rule that every foreign key value must match an existing primary key value. A row whose foreign key matches nothing is an **orphan**. The database enforces the rule by refusing to save an orphan (a sale of ProductID 12 when no product 12 exists), and by refusing to delete a parent row (a product) that still has child rows (sales) pointing at it.

A good database has…	which means
data integrity	data that is accurate, consistent and complete (referential integrity and validation protect it)
minimal redundancy	each fact stored once; a planned backup is not harmful redundancy
data security	access only for authorised users, with different rights for different users
data independence	a change to the structure does not break the programs that use it
ease of maintenance	it is easy to add, change and delete data without anomalies

6.2 Which anomaly? [5]

Circle the anomaly that each situation shows.

1	A school cannot record a new subject until at least one learner has chosen it.	**Insertion** **Deletion** **Modification**
2	When the last learner who takes Latin leaves, the fact that the school offers Latin disappears.	**Insertion** **Deletion** **Modification**
3	A teacher's cell number is typed on 40 rows. It is corrected on 38 of them.	**Insertion** **Deletion** **Modification**
4	A taxi cannot be added to the table until it has taken its first trip.	**Insertion** **Deletion** **Modification**
5	Cancelling the only booking for a hall removes the hall's hire price from the table.	**Insertion** **Deletion** **Modification**

6.3 Orphan or not? [5]

tblProduct holds the ProductIDs 1 to 8. A clerk types five new sales. Decide whether each ProductID exists, and so whether the row would be **refused** by a database that enforces referential integrity.

SaleID	ProductID	Qty	ProductID exists?	Accepted?
111	3	2		
112	9	1		
113	7	5		
114	12	3		
115	1	1		

6.4 Explain referential integrity [4]

In two or three sentences explain what referential integrity is, using tblSale and tblProduct, and what the database should do if someone tries to delete product 4 while sales of product 4 exist.

LEARN IT

Normalisation: the key, the whole key and nothing but the key

Normalisation is splitting a badly designed table into smaller tables so that each fact is stored once. We do it in three steps, each building on the one before. First write down the **dependencies**: "X → Y" means that knowing X tells you exactly one value of Y (OrderNo → OrderDate).

Normal form	The table must…	If not, then…
1NF	have one value per cell, no repeating groups of columns, and a primary key	split the lists into separate rows; choose a key (it may need two fields)
2NF	be in 1NF and every non-key field must depend on the **whole** key (this only matters when the key has two or more fields)	move each field that depends on only **part** of the key into a new table, with that part as its key
3NF	be in 2NF and no non-key field may depend on **another non-key field**	move the dependent fields into a new table whose key is the non-key field they depend on (which stays behind as a foreign key)

Worked example. Here is an order table as the shop first wrote it. Column 5 holds a list, so it is not in 1NF:

OrderNo	OrderDate	CustCell	CustName	Products ordered
1001	2026/10/01	0821112233	Mrs Dube	B01 Brown bread R16.50 × 3 C04 Candles (6) R24.00 × 2
1002	2026/10/01	0734445566	Mr Fortuin	B01 Brown bread R16.50 × 1
1003	2026/10/02	0821112233	Mrs Dube	M07 Maize meal 5kg R54.90 × 1 C04 Candles (6) R24.00 × 4
1004	2026/10/03	0617778899	Ms Peters	E02 Eggs (dozen) R49.50 × 2 M07 Maize meal 5kg R54.90 × 2

LEARN IT

Worked example: steps 1 and 2

Step 1 (1NF). Give every product ordered its own row. The key must now be **OrderNo and ProductCode together**, because neither one alone is different on every row.

`tblOrderInfo`

OrderNo	OrderDate	CustCell	CustName	ProductCode	ProductName	UnitPrice	Qty
1001	2026/10/01	0821112233	Mrs Dube	B01	Brown bread	16.50	3
1001	2026/10/01	0821112233	Mrs Dube	C04	Candles (6)	24.00	2
1002	2026/10/01	0734445566	Mr Fortuin	B01	Brown bread	16.50	1
1003	2026/10/02	0821112233	Mrs Dube	M07	Maize meal 5kg	54.90	1
1003	2026/10/02	0821112233	Mrs Dube	C04	Candles (6)	24.00	4
1004	2026/10/03	0617778899	Ms Peters	E02	Eggs (dozen)	49.50	2
1004	2026/10/03	0617778899	Ms Peters	M07	Maize meal 5kg	54.90	2

Dependencies: OrderNo → OrderDate, CustCell, CustName; CustCell → CustName; ProductCode → ProductName, UnitPrice; (OrderNo, ProductCode) → Qty.

Step 2 (2NF). OrderDate, CustCell and CustName depend on OrderNo alone, and ProductName and UnitPrice on ProductCode alone: these are **partial dependencies**. Only Qty needs the whole key. Split:

tblOrder

OrderNo PK	OrderDate	CustCell	CustName
1001	2026/10/01	0821112233	Mrs Dube
1002	2026/10/01	0734445566	Mr Fortuin
1003	2026/10/02	0821112233	Mrs Dube
1004	2026/10/03	0617778899	Ms Peters

tblProduct

ProductCode PK	ProductName	UnitPrice
B01	Brown bread	16.50
C04	Candles (6)	24.00
M07	Maize meal 5kg	54.90
E02	Eggs (dozen)	49.50

tblOrderLine

OrderNo PK	ProductCode PK	Qty
1001	B01	3
1001	C04	2
1002	B01	1
1003	M07	1
1003	C04	4
1004	E02	2
1004	M07	2

LEARN IT

Worked example: step 3 and the finished design

Step 3 (3NF). In tblOrder, CustName depends on CustCell, which is a non-key field: a **transitive dependency** (OrderNo → CustCell → CustName). Move the customer into a table of its own and keep CustCell behind as a foreign key:

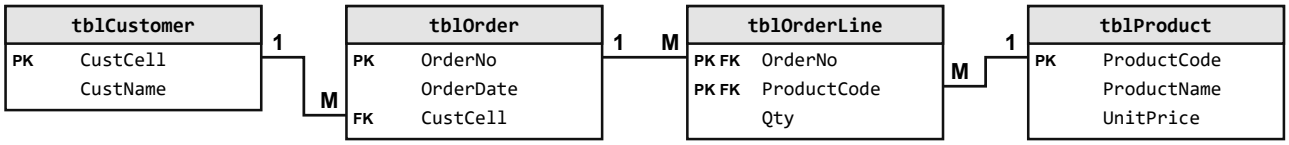
tblCustomer

CustCell PK	CustName
0821112233	Mrs Dube
0734445566	Mr Fortuin
0617778899	Ms Peters

tblOrder

OrderNo PK	OrderDate	CustCell FK
1001	2026/10/01	0821112233
1002	2026/10/01	0734445566
1003	2026/10/02	0821112233
1004	2026/10/03	0617778899

The finished design has four tables: tblCustomer, tblOrder, tblProduct and tblOrderLine. Every fact is stored once, so all three anomalies are gone, and joining the four tables gives back the original table exactly. The tblOrderLine key is made of two foreign keys: it is the **linking table** between orders and products (a many-to-many relationship made of two one-to-many ones).



6.5 Normalise: the camp bookings [8]

Learners book school camps. The primary key is **CampCode and LearnerNo together**.

tblCampBooking

CampCode	LearnerNo	CampName	CampFee	Surname	DatePaid
C1	11	Drakensberg	950	Nkosi	2026/02/10
C1	12	Drakensberg	950	Pillay	2026/02/12
C2	11	Cederberg	600	Nkosi	2026/03/01
C2	14	Cederberg	600	Adams	2026/03/05
C3	12	Robben Island	350	Pillay	2026/03/10

(a) Write two partial dependencies. (b) Write the 2NF tables with their fields and mark the PKs and FKs. (c) Is your answer also in 3NF?

6.6 Draw the ERD [5]

Draw an ERD for the finished design of Mama Dube's shop: tblCustomer, tblOrder, tblOrderLine and tblProduct. Write the fields, mark the PKs and FKs, and write **1** and **M** at the correct ends of each line.

UNIT 7 Data theory in paragraphs

Theory questions in Paper 2 reward **explanations**, not word lists. In this unit you learn how data is collected, stored and mined, how it is looked after, and what a smartphone and Big Data mean for privacy and for society. Every answer is written as a short paragraph, and you learn why that matters.

By the end of this unit I can: (tick each box when you are sure)

- ☐ name and explain the main ways that data is collected
- ☐ explain how a data warehouse differs from a database
- ☐ explain what data mining is and give a practical example
- ☐ explain how data is cared for: validation, verification, integrity and access control
- ☐ link mobile hardware to performance and battery life
- ☐ write a paragraph with a point, a reason and an example

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

7.1 A Saturday in data

Phones, cards and websites collect data about you all day, mostly without you doing anything. For each moment below write (1) **what data** is collected and (2) **who** might use it and for what. Use full sentences if you can.

7.2 Moments in the day

Complete the table.

When	What happened	Data collected and who uses it
07:10	The alarm app and the weather app open on your phone.	
09:30	You order a minibus-taxi seat through a ride app and the driver sees your location.	
11:00	You pay for airtime at the spaza shop with your bank card.	
13:00	At the supermarket you scan your loyalty card at the till.	
15:30	You search for trainers on a website, then see adverts for trainers on every other site.	
18:00	You stream a series and the app suggests the next one.	

LEARN IT

How data is collected

Data collection is gathering raw facts so that they can be stored and processed. Much of it is now automatic.

Method	How it works	Example
Forms (paper or web)	a person types or ticks the data in	registration form, online survey
RFID tags	a chip answers a reader's radio signal, so items are logged without contact or typing	library books, e-toll tag, access card

Cookies	a small file that a website saves on your device to remember you and track what you browse	a shopping cart, targeted adverts
Transaction records	every sale is saved with date, time, place and amount	card payments, till slips
Loyalty cards	the card links every purchase to one named customer over many years	a supermarket card
Sensors and apps	devices measure and send data (location, movement) all the time	a ride app, a fitness band

LEARN IT

Database, data warehouse and data mining

A **database** holds the **current** data of one system and is used for **day-to-day transactions**: rows are added, changed and deleted all day. A **data warehouse** collects **historical** data from **many** databases into one central store. Data is cleaned and put in one format before it is loaded, and it is then mostly **read**, for reports and analysis, not changed. **Data mining** is the search through such a large store for **hidden patterns** that nobody asked about, for example that customers who buy nappies on a Friday evening also buy snacks. The process is: extract the relevant data (with SQL), look for patterns, and turn them into knowledge that someone can act on, such as moving the snacks next to the nappies.

LEARN IT

Caring for data

An organisation's data is one of its most valuable assets, so it must be **accurate** (a wrong mark prints on a report: **garbage in, garbage out**), **safe** and **available**. **Validation** is the program checking that typed data is *reasonable* (a mark from 0 to 100). **Verification** checks that data matches its *source* (typing it twice, or proofreading against the form): validation cannot notice that 98 was typed as 89, verification can. **Data integrity** means the data stays correct and consistent (referential integrity, record locking). **Access control** gives each user a password and only the rights needed, and a **log** records who changed what and when. Copies kept in a second place (backups, parallel data sets) protect against loss.

LEARN IT

How to write a theory answer

Write **sentences with reasons**, not bullets. A marker may stop crediting a list after its first few items, so valid points lower down can be lost, but a paragraph is read in full. A list also hides whether you understand the idea, while a sentence with *because* or *so* proves it. A good paragraph has three parts: a **point** (what), a **reason** (why or how) and an **example** or **result**. Count the marks first: [4] means four separate points, so write at least four.
Weak: "cookies – tracking – adverts". **Strong:** "A cookie is a small file that a website saves on your device. It records which pages you visit, so the site can remember you and show adverts for things you looked at."

LEARN IT

Mobile hardware, performance and privacy

A smartphone has the same parts as a computer in miniature. Its CPU, graphics chip, memory and modem sit on a single chip, a **System on a Chip (SoC)**, to save space and power. Battery life is the main limit: the faster the processor and the brighter the screen, the faster the battery drains, and a hot phone slows itself down to cool off because it has no fan. Storage is flash memory with no moving parts, and the touchscreen is small. A phone also carries **sensors** (GPS, camera, microphone, accelerometer) and always-

on connections (Wi-Fi, 4G or 5G, Bluetooth, NFC). That makes it a very rich source of data about its owner: where they go, whom they contact and what they search. **Personal risks:** an app with too many permissions can track location or read contacts, data can be sold, and a lost phone exposes everything on it. **Business risks:** staff who use a phone for work put customer data on a device the company does not control, so a lost phone can breach privacy law; companies protect themselves with passcodes, encryption and the ability to wipe the phone remotely.

LEARN IT

Big Data and society

Big Data is data so large, fast-moving and varied that ordinary tools cannot cope. Mining it can **help** society: hospitals predict illness, banks detect fraud, and farmers use sensor data to decide when to irrigate. It can also **harm**: people lose privacy when their habits are tracked, profiles can be used to discriminate (an insurer refusing cover because of your shopping), breaches expose millions of records, and people without access to technology fall further behind. A full answer weighs both sides and says *who* benefits and *who* is affected.

7.3 A loyalty card [4]

A supermarket gives out loyalty cards. In a paragraph, explain **how** the card collects data about a customer and **how** the supermarket can use that data.

7.4 Warehouse or database? [3]

Differentiate between a **database** and a **data warehouse** in a short paragraph. Give both sides in each sentence.

7.5 Look after the data [4]

A clerk types a learner's mark of 98 as 89. Explain the difference between **validation** and **verification** using this example, and say which of them could catch the mistake.

7.6 Battery life [3]

A phone battery holds 4 500 mAh. If the phone draws an average of 600 mA, the battery lasts _____ hours (4 500 ÷ 600). If a heavy game raises the draw to 900 mA, it lasts _____ hours, which is _____ hours less.

7.7 Privacy on a mobile [5]

A learner installs a free torch app that asks for permission to use her **location**, **contacts** and **microphone**. In a paragraph, explain **two** privacy risks to the learner. Then explain **one** risk to a company whose employee uses a phone for work.

7.8 Big Data and society [5]

Discuss one benefit and one danger of mining Big Data for society. Write a paragraph.

TERM CHECKPOINT SQL, design and data

Time: 120 minutes **Total:** 74 marks **Mark:** _____ / 74

No notes, no computer. Show all your working. Write Delphi code neatly and use correct indentation.

SECTION A — SQL queries (use tblRank and tblTrip; Fare is the price per passenger) [34]

A1 [3]

A taxi association keeps these tables. The primary key of tblRank is RankID, and RankID is a foreign key in tblTrip. **Question:** write a query that shows the TripID, Fare and Passengers of the trips with **10 or more passengers**, with the **highest fare first**.

`tblRank`

RankID PK	RankName	Manager
1	Bellville	Mr Fortuin
2	Parow	Mrs Jansen
3	Goodwood	Mr Mthembu
4	Elsies River	Ms Daniels

`tblTrip`

TripID PK	RankID	Fare	TripDate	Passengers
201	1	28.00	2026/10/01	14
202	2	22.50	2026/10/01	9
203	1	31.00	2026/10/02	15
204	3	19.00	2026/10/02	6
205	2	24.00	2026/10/03	12
206	1	27.00	2026/10/04	11
207	3	21.00	2026/10/04	8
208	2	26.00	2026/10/05	13
209	1	30.00	2026/10/05	10
210	3	18.00	2026/10/06	5

A2 [4]

Show the TripID, RankID and TripDate of the trips from **ranks 1 and 3** that took place on or after **3 October 2026**. Use **IN** for the ranks.

A3 **[3]**

For the trips of **rank 2**, show the TripID and a calculated column called **Income** (Fare multiplied by Passengers).

A4 **[4]**

Write **one** query that gives the number of trips (**NumTrips**), the total number of passengers (**TotalPax**) and the highest fare (**MaxFare**).

A5 **[5]**

Work out the **result** of this query on the tables above.

```SQL
SELECT RankID, COUNT(*) AS NumTrips,
ROUND(AVG(Fare), 2) AS AvgFare
FROM tblTrip
GROUP BY RankID
HAVING AVG(Fare) > 20;
```

RankID	NumTrips	AvgFare

A6 **[4]**

Write a query that shows the **RankName**, **TripDate** and **Fare** of every trip with a fare **above R25**, in order of date. Join the tables with WHERE.

A7 **[5]**

Show each **RankName** and the money its trips earned (**Fare × Passengers** added up, called **Takings**), the **biggest** amount first.

A8 **[6]**

Write the SQL for each of these three changes.
(a) Add a trip for **rank 4**: fare R25.50, **7 October 2026**, 7 passengers (TripID is an AutoNumber).
(b) **Add R2** to the fare of every trip of **rank 3**.
(c) **Delete** every trip with **fewer than 6** passengers.

SECTION B — SQL in Delphi (the TADOQuery is called qryTrip) [14]

B1 **[3]**

Circle the method that runs each statement.

1	SELECT TripID FROM tblTrip WHERE RankID = :pRank	**Open**	**ExecSQL**
2	DELETE FROM tblTrip WHERE TripID = 210	**Open**	**ExecSQL**
3	UPDATE tblTrip SET Fare = Fare + 2	**Open**	**ExecSQL**

B2 **[3]**

The user types **2** in edtRank. (a) Write the exact text stored in sSQL. (b) Write the TripID and Fare of the rows returned (use the tables in question A1).

Delphi

```
sRank := edtRank.Text;
sSQL := 'SELECT TripID, Fare ' +
        'FROM tblTrip ' +
        'WHERE RankID = ' + sRank + ' ' +
        'AND Fare >= 24 ' +
        'ORDER BY Fare';
```

B3 **[8]**

edtMinFare holds a fare. When **btnShow** is clicked, use a query with **one parameter** (**pFare**) to list in **memTrips** the TripID and fare of every trip whose fare is **above** that fare, highest fare first, each line as *TripID: Rfare*. Show the number of trips in **lblNum**.

SECTION C — Database design and normalisation [12]

C1 **[3]**

A teacher stores learner subjects like this:

tblSubjects

LearnerNo	Surname	Subjects
31	Zulu	IT, Maths, English
32	Pillay	Maths, Science

(a) Why is the table **not in 1NF**? (b) Rewrite the data so that it **is** in 1NF (write the rows).

C2 **[9]**

A school keeps the results of its athletics day in one table. The primary key is **LearnerNo and EventCode together**.

`tblEntry`

LearnerNo	Surname	HouseCode	HouseHead	EventCode	EventName	Result
21	Zulu	GRE	Mr Daniels	E1	100 m	13.2 s
21	Zulu	GRE	Mr Daniels	E3	Long jump	4.85 m
22	Moodley	RED	Ms Pretorius	E1	100 m	12.8 s
23	Smit	GRE	Mr Daniels	E2	800 m	2:41
23	Smit	GRE	Mr Daniels	E3	Long jump	4.10 m
24	Sibiya	RED	Ms Pretorius	E2	800 m	2:35

(a) Name **one partial dependency** and **one transitive dependency** in the table. (b) Write the tables of a **3NF** design with all their fields, and mark every primary key (PK) and foreign key (FK).

SECTION D — Data theory (answer in sentences) [14]

D1 **[4]**

A supermarket gives its customers a loyalty card. In a paragraph, explain **two ways** in which the shop collects data about a customer and how it can **mine** that data.

D2 **[3]**

Differentiate between a database and a data warehouse.

D3 **[3]**

Explain the difference between **validation** and **verification**, with an example of each.

D4 **[4]**

A delivery rider's smartphone drains its battery by lunchtime and becomes hot when he uses the map. (a) **Explain** two hardware reasons why this happens. (b) Give **one** privacy risk of the rider's phone to the delivery company.

COMING UP NEXT TERM · warm-up

A taxi and its blueprint

In Term 2 you will write programs made of **objects**. Before any code, think about minibus taxis.
(a) A factory builds hundreds of taxis from **one design drawing**. List four things **every** taxi **has** (for example, a number plate).
(b) List four things a taxi **can do**, or that can be done to it (for example, pick up a passenger).
(c) Two taxis are built from the same drawing. Fill in what is **the same** and what is **different** for your four things.
(d) Why is one design drawing better than drawing a new plan for every taxi?
(e) A passenger cannot change the reading on the odometer; only driving does that. Why is it good that some things inside the taxi are hidden from the passengers?